

Uwierzytelnianie klienta SSH za pomocą kluczy prywatnych

Filip Świniarski 197749
Nikodem Miłuch 197922
Tomek Lisowski 197749

1. Wstęp

SSH (Secure Shell) to protokół sieciowy służący do bezpiecznego zdalnego logowania się i zarządzania systemami komputerowymi przez sieć. Umożliwia szyfrowaną komunikację między klientem a serwerem, chroniąc dane przed podsłuchem i atakami. Najczęściej używany jest do administrowania serwerami Linux/Unix z poziomu terminala.

2. Wstępna konfiguracja SSH

2.1 Instalacja pakietu OpenSSH.

Na początku założymy, że nasz serwer jest maszyną z systemem Linux, połączoną w tej samej sieci co nasz klient. Pierwszym krokiem jest sprawdzenie, czy na naszym serwerze posiadamy program SSH w wersji serwerowej. By to sprawdzić, należy wykonać na serwerze komendę:

```
$ sshd -V
```

Powinna się wyświetlić wersja OpenSSH. W przypadku braku tego pakietu na serwerze, wymagana jest jego instalacja, za pomocą systemu zarządzania pakietami danego systemu. Dla systemów bazowanych na Debian będzie to:

```
$ sudo apt-get install openssh-server
```

Po instalacji, pierwsza komenda powinna zadziałać. Należy również sprawdzić, czy program nie jest w folderze **/usr/sbin**, który może nie być dołączony do zmiennej PATH.

2.2 Plik konfiguracyjny

Konfiguracja serwera SSH polega na zmianach w pliku **/etc/ssh/sshd_config**. Aby go zmodyfikować, trzeba wykonać komendę:

```
$ sudo nano /etc/ssh/sshd_config
```

W tym momencie, trzeba znaleźć zakomentowaną opcję **PubkeyAuthentication**, którą należy odkomentować i zmienić jej wartość na yes. Pozwoli to na uwierzytelnianie klientów ssh za pomocą kluczy.

#PubkeyAuthentication no

Zamienić na

PubkeyAuthentication yes

Odkomentować(usunąć znak # z początku) należy również opcję **AuthorizedKeysFile**:

AuthorizedKeysFile ./ssh/authorized_keys

Reszta konfiguracji zostanie przeprowadzona w późniejszych krokach. Teraz należy tylko zresetować daemon sshd poprzez:

\$ sudo systemctl restart sshd

3. Generowanie kluczy

Wszystkie poniższe kroki powinny zostać wykonane na komputerze klienta. Pakiet klienta OpenSSH jest domyślnie zainstalowany na Windows 10/11 oraz większości dystrybucji Linux.

3.1 Algorytmy generowania kluczy SSH

Do powszechnie wykorzystywanych algorytmów generujących bezpieczne klucze SSH należą algorytmy:

1. Rivest-Sharim-Adleman (RSA) – powszechnie wspierany i przebadany, oparty na faktoryzacji dużych liczb pierwszych.
2. Ed25519 – algorytm szybszy niż RSA, oparty na krzywych eliptycznych (Curve25519), rozmiar klucza w przeciwieństwie do RSA jest stały i wynosi 256 bitów. Jest to zalecany algorytm generowania kluczy.
3. Elliptic Curve Digital Signature Algorithm (ECDSA) – zasada tworzenia jak w Ed25519, klucz może mieć różne rozmiary.

3.2 Generowanie kluczy w systemie Linux

Generowanie pary kluczy SSH w systemie Linux można wykonać korzystając z komendy:

\$ ssh-keygen -t {algorytm} -b {rozmiar klucza} -C {komentarz}

Podany rozkaz można modyfikować o rodzaj algorytmu np. *rsa*, *ed25519* itp.

3.2 Generowanie kluczy na systemie Windows

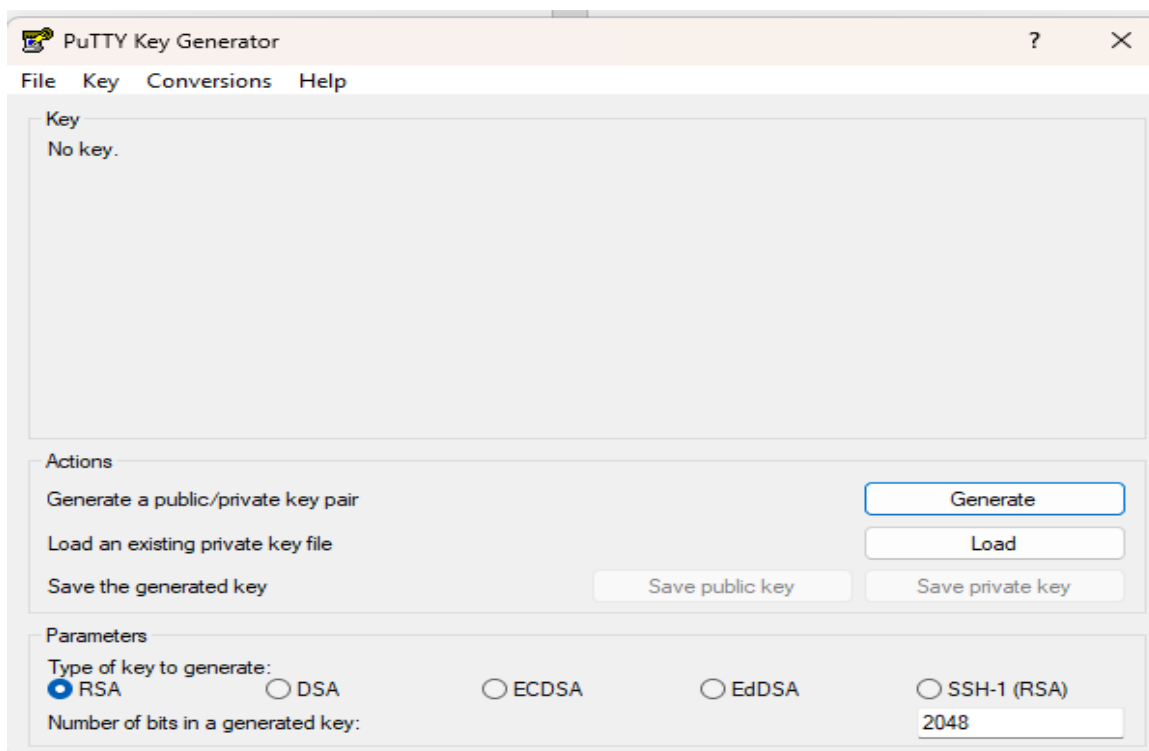
Generowanie klucza w systemie Windows z wykorzystaniem CMD przebiega analogicznie jak w systemie Linux.

```
$ ssh-keygen -t {algorytm} -b {rozmiar klucza} -C {komentarz}
```

3.3 Generowanie kluczy za pomocą PuttyGen

Alternatywą do generowania pary kluczy z wykorzystaniem terminala lub wiersza poleceń jest wykorzystanie aplikacji PuttyGen, dostępnej w każdym systemie operacyjnym.

Po zainstalowaniu aplikacji należy kliknąć przycisk "Generate", który rozpocznie proces tworzenia nowej pary kluczy, z wykorzystaniem losowości ruchów myszki użytkownika. Następnie należy zabezpieczyć klucz w polu "Key passphrase" hasłem.



Wygenerowany klucz prywatny należy przenieść (jeżeli z jakiegoś powodu się tam nie znalazł) do pliku .ssh.

W systemie Linux:

```
$ ~/.ssh
```

W systemie Windows:

C:\Users\"UserName"\.ssh

4.Przesłanie kluczy publicznych do serwera

Wszystkie poniższe kroki powinny zostać wykonane na komputerze klienta. Do tych kroków będzie potrzebna nazwa użytkownika (nazywana dalej **user**) do którego będziemy chcieli się logować oraz adres IP serwera na sieci lokalnej (nazywana dalej **adres**).

4.1 Przesyłanie kluczy publicznych w systemie Linux

Wygenerowane wcześniej klucze, powinny znajdować się w folderze /.ssh . By przekopiować klucz, wystarczy użyć dedykowanego narzędzia ssh-copy-id :

```
$ ssh-copy-id user@adres
```

Powyższa komenda skopiuje klucz publiczny do folderu **.ssh/authorized_keys** danego użytkownika. Jeśli chcemy wybrać, który klucz ma zostać przesłany (domyślne jest to id_rsa.pub) możemy skorzystać z opcji **-i**.

```
$ ssh-copy-id -i ~/.ssh/nazwa_klucza.pub user@adres
```

4.2 Przesyłanie kluczy publicznych w systemie Windows

Aby wysłać na serwer klucz publiczny wygenerowany w systemie Windows należy skopiować klucz korzystając z komendy:

```
type %env:USERPROFILE\.ssh\nazwaklucza.pub
```

Lub manualnie skopiować Eksploratora Plików. Następnie wymagane jest zalogowanie się na serwer wykorzystując komendę:

```
$ssh user@adres_ip
```

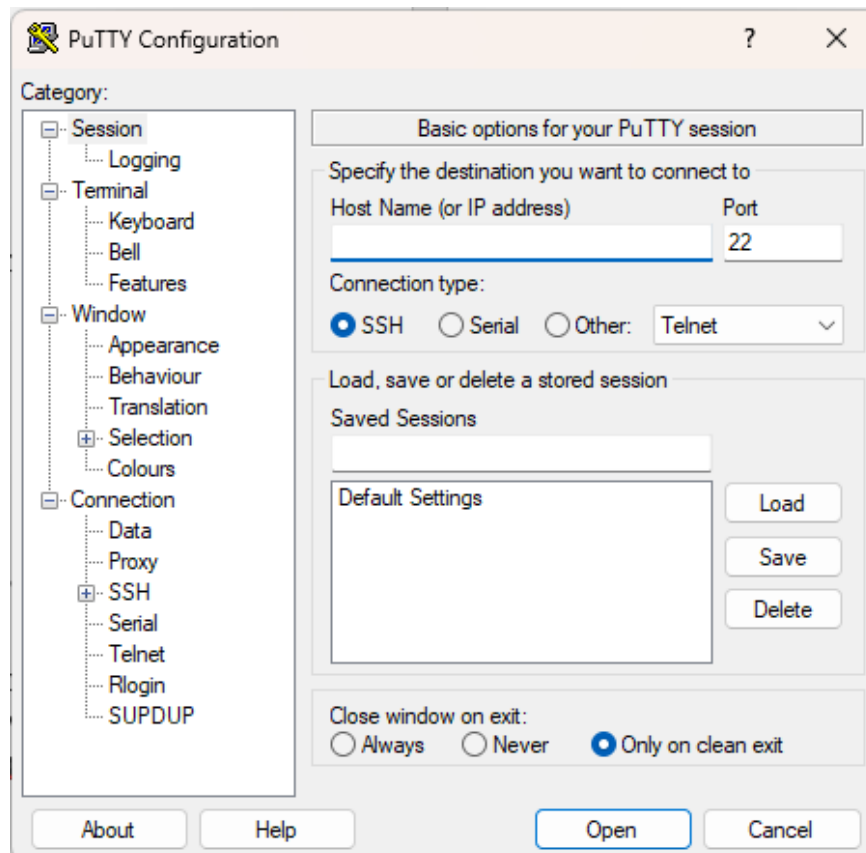
Po pomyślnym zalogowaniu do folderu *authorized_keys* należy utworzyć plik z rozszerzeniem *.pub* (np. id_rsa.pub) i skopiować do niego zawartość klucza publicznego. Następnie należy wylogować się z serwera. Serwer powinien być gotowy do zalogowania się poprzez SSH, wykorzystując komendę:

```
$ssh user@adres_ip
```

4.3 Przesyłanie kluczy publicznych z wykorzystaniem Putty

W przypadku skorzystania z Putty i PuttyGen różnicę stanowi kopiowanie klucza i łączenie przeprowadzane przy pomocy GUI tych aplikacji oraz rozszerzenia kluczy

publicznego i prywatnego (.ppk dla prywatnego i brak rozszerzenia dla publicznego). W polu Host Name należy podać nazwę serwera lub jego IP.



Najszybszą metodą na udostępnienie klucza publicznego jest postępowanie analogiczne jak w punkcie 4.2. Z racji tego, że klucz generowany przez PuttyGen nie ma rozszerzenia, zawartość tego klucza musi zostać skopiowana do nowego pliku z rozszerzeniem .pub

Aby połączenie mogło wystąpić należy wskazać miejsce przechowywania klucza prywatnego (rozszerzenie .ppk). W tym celu należy wybrać **Connection > SSH > Auth > Browse** i wskazać klucz prywatny. Aby nawiązać połączenie należy wrócić do zakładki **Session** i wybierać opcję **Open**.

5.Konfiguracja SSH

Po poprzednich krokach możemy zmienić konfigurację serwera SSH, tak by była bardziej (z punktu widzenia cyberbezpieczeństwa) bezpieczna. By edytować konfigurację SSH, należy wykonać następującą komendę na serwerze:

```
$ sudo nano /etc/ssh/sshd_config
```

Pierwszą opcją, którą powinniśmy wyłączyć jest możliwość uwierzytelniania się hasłem. Dzięki temu można ominąć możliwe ataki typu brute-force na nasz serwer. W pliku należy znaleźć zakomentowaną linię PasswordAuthentication i zmienić jej wartość na no:

```
#PasswordAuthentication yes
```

Zamienić na

```
PasswordAuthentication no
```

Kolejnym krokiem zwiększającym bezpieczeństwo jest wyłączenie logowania się na konto root poprzez SSH. Pozwoli to uniknąć sytuacji, w której atakujący próbuje odgadnąć hasło (lub klucz) do konta root, które jest domyślnie obecne na każdym systemie Linux. Aby to zrobić, należy znaleźć w pliku konfiguracji linię:

```
#PermitRootLogin yes
```

I zamienić na:

```
PermitRootLogin no
```

Dzięki temu nawet ktoś kto zna dane dostępowe roota, nie będzie mógł się zalogować przez SSH.

Warto też zmienić domyślny port SSH o numerze 22 na inny z zakresu 1024–65535. W pliku konfiguracji należy zmienić linię:

```
#Port 22
```

na np.

```
Port 9822
```

oczywiście po tej zmianie należy połączyć się z serwerem podając już nowy port.

Oprócz tego możemy wyłączyć przekazywanie X11. Aby to zrobić należy zmienić opcję:

```
X11Forwarding yes
```

na:

```
X11Forwarding no
```

Teraz należy zresetować daemon sshd poprzez:

```
$ sudo systemctl restart sshd
```

Można przetestować konfigurację, poprzez podłączenie się do serwera z komputera klienta, od którego został przesłany klucz publiczny. Po połączeniu powinniśmy mieć dostęp do serwera bez wpisywania hasła:

5.1 Połączenie się z systemu Linux i Windows

Połączenie dla systemu Linux i Windows wygląda bliźniaczo. W Windows należy włączyć program cmd, a w Linux terminal i wpisać następującą komendę:

```
$ ssh user@adres
```

Można skorzystać z opcji `-p` by wskazać port oraz opcji `-i` aby wskazać który klucz ma zostać użyty do uwierzytelnienia połączenia (domyślne ssh sprawdza wszystkie znajdujące się w folderze `/.ssh`)

```
$ ssh -p PORT -i ~/.ssh/nazwa_klucza
```

6. Transfer plików z uwierzytelnianiem kluczami prywatnymi

Do bezpiecznego transferu plików z uwierzytelnianiem kluczami prywatnymi najczęściej używa się narzędzi takich jak:

- **SCP (Secure Copy)** to narzędzie do bezpiecznego kopiowania plików pomiędzy lokalnym a zdalnym systemem przez SSH. Umożliwia szybki transfer pojedynczych plików lub całych katalogów.

```
scp -i ~/.ssh/id_rsa plik.txt user@remote_host:/ścieżka/docelowa/
```

- **SFTP(Secure FTP)** to interaktywny protokół do bezpiecznego przesyłania plików przez SSH. Działa podobnie do FTP, ale wykorzystuje szyfrowanie SSH. Umożliwia nawigację po katalogach, pobieranie i wysyłanie plików.

```
sftp -i ~/.ssh/id_rsa user@remote_host
```

ls - Wyświetla pliki i foldery na serwerze

cd - Przechodzi do folderu na serwerze

get - Pobiera plik z serwera na komputer

put - Wysyła plik z komputera na serwer

rm - Usuwa plik na serwerze

mkdir - Tworzy folder na serwerze

exit / bye - Kończy sesję SFTP

gdzie:

-i ~/.ssh/id_rsa wskazuje na plik klucza prywatnego, **user@remote_host** to użytkownik i adres serwera, natomiast **/ścieżka/docelowa** to miejsce zapisania pliku

- **WinSCP** – graficzny klient SFTP dla systemu MS Windows korzystający z SSH.

Do transferu plików w **WinSCP** potrzebujemy klucza prywatnego skonwertowanego do formatu **.ppk**, co możemy zrobić przy użyciu PuTTYgen oraz klucza publicznego umieszczonego na serwerze w pliku **~/.ssh/authorized_keys**.

Po otwarciu WinSCP ustawiamy protokół SFTP i kolejno wprowadzamy:

- Host name
- User name
- Nasz klucz prywatny

